

SRS

Software Requirements Specification (SRS)

Blockchain Voting System (BVS)

Cryptographic E-Voting & Decentralized Ledger Module

Document Version: 1.0

Prepared By: Aditya Yadav

Department: B.Sc Computer Science

Institution: Thakur Ramnarayan College of Arts & Commerce (TRCAC)

Academic Year: 2026-2027

Date: August 2026

Status: Final Draft

Table of Contents

1. Introduction
2. Overall Description
3. User Roles & Characteristics
4. Functional Requirements
5. Non-Functional Requirements
6. System Constraints
7. External Interface Requirements
8. Data Requirements

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the **Blockchain Voting System (BVS) — Cryptographic E-Voting & Decentralized Ledger Module**. It is intended as a reference for the development team, evaluators, instructors, and any stakeholders involved in reviewing, testing, deploying, or auditing the system.

1.2 Scope

The **Blockchain Voting System (BVS)** is a secure, decentralized web-based platform designed to conduct tamper-evident, cryptographically verifiable elections across general organizations, academic institutions, and enterprise bodies. The system supports multiple election domains and configurable voter electorates:

Election Category	Configurable Electorates & Sub-divisions
Institutional / Campus Elections	Student Council, Department Representative, Senate, Faculty Committee
Organizational / Corporate Ballots	Board of Directors, Shareholder Voting, Executive Committee, Union Elections
Community & Guild Elections	Association Leadership, Civic Panels, Club Executives, Regional Delegates
General E-Voting & Polls	Multi-candidate Polls, Referendums, Resolution Approvals, Policy Feedback
Academic Departments (TRCAC)	B.Sc CS, B.Sc IT, BA, BAMMC, BMS, B.Com, BAF (FY, SY, TY cohorts)

The system has three primary user roles: **Admin**, **Voter**, and **Observer / Auditor**, each equipped with dedicated dashboards, security boundaries, and authorization privileges.

1.3 Definitions & Acronyms

Term	Definition
BVS	Blockchain Voting System
PoW	Proof-of-Work consensus algorithm enforcing computational block mining
ECDSA	Elliptic Curve Digital Signature Algorithm (SECP256R1 curve)
AES-256 GCM	Advanced Encryption Standard (256-bit) with Galois/Counter Mode authenticated encryption
SHA-256	Secure Hash Algorithm 256-bit cryptographic digest function
Merkle Tree	Cryptographic binary hash tree aggregating all transactions within a block
JWT	JSON Web Token — stateless cryptographic bearer token for session authentication
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
TRCAC	Thakur Ramnarayan College of Arts & Commerce

1.4 References

- Project Codebase: blockchain-voting-system (GitHub Repository)
- NIST Special Publication 800-38D (Recommendation for GCM Block Cipher Mode)
- SEC 2: Recommended Elliptic Curve Domain Parameters (SECP256R1)
- System Architecture & Security Specs: SYSTEM_ARCHITECTURE.md

1.5 Overview

The BVS is architected into two decoupled logical tiers:

9. **Frontend** — A responsive React 19 + Vite Single Page Application (SPA) styled with Tailwind CSS
10. **Backend** — An asynchronous FastAPI (Python 3.12) REST API connected to database and Custom Blockchain Engine

2. Overall Description

2.1 Product Perspective

The Blockchain Voting System is a standalone e-voting solution replacing physical ballot boxes and centralized, opaque databases. The system can be deployed on local infrastructure or cloud environments.

The system integrates with:

- **SQLite / PostgreSQL** — relational database for relational application metadata
- **Custom Python Blockchain** — decentralized SHA-256 ledger engine with Merkle roots
- **FastAPI WebSockets** — real-time vote distribution and block broadcast updates

2.2 Product Functions (High-Level)

The following high-level capabilities are supported by the system:

- 11. User Account Management — Admin creates accounts; role-based login (Admin, Voter, Observer)
- 12. Election Lifecycle Management — Admin creates, schedules, activates, and closes elections
- 13. Candidate Nomination Studio — Admin registers candidates with party affiliations and manifestos
- 14. 1-Click Encrypted Voting — Vote choices encrypted via AES-256 GCM before ledger submission
- 15. Anti-Double Voting Enforcement — Single vote per registered voter strictly enforced at database and chain level
- 16. Digital Signature Signing — Transactions signed using single-use voter ECDSA key pairs
- 17. Proof-of-Work Block Mining — Mined blocks aggregate queued transactions meeting SHA-256 difficulty targets
- 18. Downloadable QR Receipt — Generates printable receipt containing receipt hash, block index, and tx hash
- 19. Bulk Voter Enrollment — Admin bulk-enrolls voters via CSV file upload
- 20. Blockchain Explorer — Public block browser, transaction lookup, and 1-click ledger tamper auditor
- 21. AI Fraud Radar — Analyzes vote velocity bursts (<2s), duplicate IP clusters, and double-voting threat index

2.3 User Classes

User Class	Description
Admin	Full administrative control. Manages elections, candidate profiles, bulk voter enrollment, and audit logs.
Voter	Casts encrypted ballots in active elections, downloads QR receipts, and verifies receipt hashes on the ledger.
Observer / Auditor	Read-only transparency access. Watches live Recharts vote tallies, inspects block explorer, verifies chain integrity, and views AI fraud alerts.

2.4 Operating Environment

- **Client Side:** Modern web browser (Chrome, Firefox, Edge, Safari)
- **Server Side:** Python 3.12+, FastAPI 0.111+, Uvicorn 0.30+
- **Database:** SQLite (local development) / PostgreSQL (cloud-hosted production)
- **Frontend Build Tool:** Vite 5.x / 7.x
- **Deployment:** Docker, Docker Compose, or Vercel / Cloud VM deployment

2.5 Design and Implementation Constraints

- Frontend is built with **React 19** and **Tailwind CSS 3.4**
- Backend uses Python 3.12 with FastAPI asynchronous REST framework
- JWT tokens expire in **24 hours** for session security

- File uploads for voter CSV import are limited to **10 MB** per file
- Allowed file type for bulk voter import: CSV
- Rate limiting is applied to authentication and vote submission routes
- CORS is strictly configured to allow only registered frontend domains

2.6 Assumptions and Dependencies

- Users have stable internet connectivity.
- Every voter possesses a valid unique email and account credentials.
- Database and custom blockchain ledger services remain active during polling.
- Cryptographic secret keys for vote encryption and JWT signing are properly secured in environment variables.

3. User Roles & Characteristics

3.1 Admin

- **Technical proficiency:** Moderate; expected to use a web-based admin dashboard
- **Access level:** Full system access
- **Typical tasks:** Create elections, add candidates, bulk-enroll voters, monitor metrics, view audit logs
- **Authentication:** Stored in User table (role = 'admin')

3.2 Voter

- **Technical proficiency:** Basic to moderate
- **Access level:** Scoped to assigned active elections and personal ballot casting
- **Typical tasks:** View candidates, cast encrypted vote, download QR receipt, verify receipt hash
- **Authentication:** Stored in User table (role = 'voter')

3.3 Observer / Election Auditor

- **Technical proficiency:** Basic to moderate
- **Access level:** Read-only access to public transparency metrics
- **Typical tasks:** View live results, inspect block ledger, audit cryptographic integrity, view AI fraud alerts
- **Authentication:** Stored in User table (role = 'observer')

Actor	Use Cases
Admin	Manage Users, Elections, Candidates, Bulk CSV Import, Audit Logs, System Health
Voter	View Active Elections, Cast Encrypted Vote, Download QR Receipt, Verify Receipt Hash
Observer / Auditor	Live Standings, Audit Blockchain Ledger, Chain Verification, AI Fraud Alerts

4. Functional Requirements

4.1 Authentication & Authorization Module

FR-AUTH-01: User Login

- **Description:** The system shall allow admin, voters, and observers to log in using email/username and password.
- **Input:** Email/Username, Password
- **Process:** System searches user record and verifies password using bcrypt. Returns a JWT access token.
- **Output:** JWT access token + user profile data
- **Token Expiry:** 24 hours
- **Error Cases:** Invalid credentials -> HTTP 400

FR-AUTH-02: Admin User Creation

- **Description:** Only authenticated admins can create new user accounts.
- **Input:** Username, email, password, role (voter/observer)
- **Process:** Admin-authenticated endpoint creates record in User table
- **Validation:** Unique email and username required

FR-AUTH-03: Logout

- **Description:** Users can log out, clearing local storage JWT token.
- **Process:** Token removed from client storage; subsequent requests require re-authentication.

FR-AUTH-04: Token Verification

- **Description:** Protected routes verify the Bearer token before proceeding.
- **Process:** JWT decoded -> user looked up by ID -> user role validated

FR-AUTH-05: Password Change

- **Description:** Users can change their own passwords securely.
- **Input:** Current password, new password (min 6 characters)
- **Process:** Verify current password -> bcrypt hash new password -> save to database

4.2 Admin Dashboard Module

FR-ADMIN-01: Dashboard Overview

- **Description:** Admin home page displays aggregated metrics:
 - Total Elections
 - Active Elections
 - Total Registered Voters
 - Total Blockchain Votes Mined
 - Blockchain Health & Mining Status
- **Data Source:** GET /api/v1/admin/metrics

FR-ADMIN-02: Election Management

- **Description:** Admin can create, edit, activate, schedule, and close elections.
- **Operations:** List all elections; create new election; update status (draft, active, closed)
- **Route:** POST /api/v1/elections, PUT /api/v1/elections/:id/status

FR-ADMIN-03: Candidate Nomination Studio

- **Description:** Admin can register candidates for an election with full metadata.
- **Fields:** Candidate Name, Party / Affiliation, Manifesto Description, Avatar URL
- **Route:** POST /api/v1/elections/:id/candidates

FR-ADMIN-04: Bulk Voter Enrollment

- **Description:** Admin can upload a CSV file to bulk-create voter accounts.
- **CSV Fields:** Email, Username, Password
- **Route:** POST /api/v1/admin/import-voters-csv

FR-ADMIN-05: Immutable Audit Logs

- **Description:** Admin can view and download immutable audit logs of all user actions.
- **Route:** GET /api/v1/admin/audit-logs

4.3 Voter Dashboard Module

FR-VOTER-01: Active Elections List

- **Description:** Voter home page displays all active elections and candidate profiles.
- **Route:** GET /api/v1/elections

FR-VOTER-02: 1-Click Encrypted Voting

- **Description:** Voter selects candidate and casts vote encrypted with AES-256 GCM and signed with single-use ECDSA key pair.
- **Route:** POST /api/v1/votes/cast

FR-VOTER-03: Anti-Double Voting Enforcement

- **Description:** System blocks duplicate vote attempts for the same election and logs violation to security audit trail.

FR-VOTER-04: Downloadable QR Receipt

- **Description:** Generates printable QR receipt containing receipt hash, block index, and transaction hash.
- **Route:** GET /api/v1/votes/my-receipts

FR-VOTER-05: Receipt Verification Desk

- **Description:** Voters can verify any receipt hash against the public blockchain ledger.
- **Route:** GET /api/v1/votes/verify-receipt/:hash

4.4 Custom Blockchain Module

FR-CHAIN-01: Proof-of-Work Block Mining

- **Description:** Automatically packages queued transactions into blocks meeting SHA-256 Proof-of-Work difficulty target (leading zero prefix).

FR-CHAIN-02: Merkle Tree Generation

- **Description:** Computes cryptographic binary Merkle Root hash from all transaction hashes in a block.

FR-CHAIN-03: Ledger Explorer & Tamper Audit

- **Description:** Public explorer endpoint allowing block browsing, transaction lookup, and 1-click full chain validation.
- **Route:** GET /api/v1/blockchain/blocks, GET /api/v1/blockchain/verify-chain

4.5 AI Fraud Radar & Anomaly Detection Module

FR-AI-01: Fraud Risk Index & Velocity Analysis

- **Description:** Analyzes vote velocity bursts (<2s), duplicate IP clusters, and double-voting attempts to output a unified threat score (0-100%).
- **Route:** GET /api/v1/ai/fraud-analysis/:election_id

4.6 Observer Transparency & Analytics Module

FR-OBSERVER-01: Live Standings & Vote Tallies

- **Description:** Real-time display of current vote counts, percentages, and leading candidates per election.
- **Route:** GET /api/v1/observer/live-results/:election_id

FR-OBSERVER-02: Recharts Vote Distribution Visualizer

- **Description:** Interactive bar and pie charts rendering vote distribution and voter turnout percentages.

FR-OBSERVER-03: Audit Report PDF Export

- **Description:** Allows election observers to export official election tally and audit certificates.

5. Non-Functional Requirements

5.1 Performance

Metric	Requirement
API Response Time	< 500 milliseconds for standard data endpoints under normal load
File Upload	Handles CSV files up to 10 MB; uploads complete within 10 seconds
Block Mining	PoW block mined and synced to database within 2 seconds
Real-Time Broadcast	WebSocket vote updates pushed to observers within 1 second
Concurrent Users	Supports at least 500 concurrent voters in production configuration

5.2 Security

Requirement	Detail
Authentication	JWT Bearer tokens, 24-hour expiry
Password Storage	bcrypt with salt rounds = 10
Vote Encryption	AES-256 GCM symmetric vote payload encryption
Digital Signatures	ECDSA SECP256R1 asymmetric key pair signatures
Rate Limiting	Auth routes: strict limiter; Vote routes: velocity limiter
CORS	Whitelist-only: localhost:3000 and production domain URL
Input Validation	Pydantic schemas for all incoming request payloads
SQL Injection Defense	SQLAlchemy ORM parameterized queries

5.3 Reliability

- System maintains **99.9% uptime** during voting windows. Immutable blockchain ledger guarantees zero vote loss or record modification.

5.4 Usability

- Responsive layout built with Tailwind CSS — supports desktop, tablet, and mobile browsers with dark/light mode toggle.

5.5 Scalability

- Stateless JWT authentication and relational database indexing support horizontal backend scaling.

5.6 Maintainability

- Clear architectural separation of concerns: API Routes -> Services -> Blockchain Core -> Database Models.

6. System Constraints

- 22. Blockchain Proof-of-Work difficulty target is set to 2 leading zeros for lightweight execution.
- 23. File uploads for voter CSV enrollment are restricted to CSV format under 10 MB.
- 24. Single vote policy strictly enforced per voter per election via compound database uniqueness.
- 25. Vote payloads are encrypted at rest using server-side AES-256 GCM secret key.
- 26. Token Blacklist operates in-memory with client-side localStorage cleanup.
- 27. Real-time push updates require WebSocket client compatibility.

7. External Interface Requirements

7.1 User Interfaces

Page	Role	Description
Login / Register	Public	Authentication portal with role selector (Admin, Voter, Observer)
Admin Dashboard	Admin	Sidebar with Overview, Elections CRUD, Candidates, CSV Import, Audit
Voter Dashboard	Voter	Active Ballots, 1-Click Encrypted Voting, QR Receipts, Verification
Observer Dashboard	Observer	Live Recharts standings, Vote Distribution, Turnout Percentages
Blockchain Explorer	Public	Mined Blocks, Transaction Lookup, 1-Click Tamper Auditor
AI Fraud Radar	Admin/Observer	Real-time velocity spikes, duplicate IP clusters, Threat Index

7.2 Hardware Interfaces

- Standard web-enabled client hardware (PC, Laptop, Tablet, Smartphone)

7.3 Software Interfaces

System	Interface
SQLite / PostgreSQL	SQLAlchemy ORM connection protocol
FastAPI WebSockets	Real-time bidirectional event manager
Vercel / Docker	Static CDN hosting and containerized ASGI backend

7.4 Communication Interfaces

- **Protocol:** HTTPS (production), HTTP (local development)
- **Data Format:** JSON (REST API & WebSockets)
- **Auth Header:** Authorization: Bearer <JWT_TOKEN>

8. Data Requirements

8.1 Data Entities

Entity	Collection / Table	Key Fields
User	users	id, email, username, hashed_password, role, is_active, created_at
Election	elections	id, title, description, status, start_time, end_time, created_by
Candidate	candidates	id, election_id, name, party, manifesto, avatar_url, vote_count
Vote	votes	id, user_id, election_id, candidate_id, voter_hash, tx_hash, block_index, receipt_hash
Block	blocks	id, index, timestamp, previous_hash, hash, nonce, merkle_root, signature
Transaction	transactions	id, tx_hash, block_index, election_id, voter_hash, encrypted_vote, signature
Audit Log	audit_logs	id, user_id, user_email, action, details, ip_address, timestamp

8.2 Data Retention

- Security audit logs and voter activity logs are timestamped and preserved for election verification.
- Blockchain blocks and vote transactions are immutable — once mined into the ledger, records persist permanently.

8.3 Data Integrity

- Unique email and username constraints on User collection prevent duplicate accounts.
- Unique compound constraint on Vote (user_id, election_id) guarantees single vote enforcement.
- Unique cryptographic hashes on Block (index, hash) and Transaction (tx_hash) guarantee tamper-evident ledger integrity.